Introduction

This assignment implements the payment calculator, tests its behaviour, compares actual results with expected results, justifies the final design, and identifies improvements that could be made if the co-operative's requirements change.

TASK 9 — Implement and Test the Calculator

Complete program

```
def calculate_payment(milk_quantity, rate_per_litre):
    if milk_quantity < 0:
        raise ValueError("Milk quantity cannot be negative")
    if rate_per_litre < 0:
        raise ValueError("Rate cannot be negative")
    return milk_quantity * rate_per_litre

def main():
    try:
        quantity = float(input("Enter milk quantity in litres: "))
        rate = float(input("Enter rate per litre: "))
        payment = calculate_payment(quantity, rate)
        print("Total payment: Rs.", format(payment, ".2f"))
    except ValueError as error:
        print("Error:", error)

if __name__== "__main__":
    main()
```

The calculate_payment() function checks for negative values and returns the multiplication result. The main() function reads inputs, calls the calculation function, and displays the result to two decimal places. The try/except block handles non-numeric input and validation errors.

Test cases

Test	Quantity	Rate	Expected result	Purpose
1	12.5	38.0	■475.00	Normal decimal input
2	10	38	■380.00	Whole-number input
3	0	38	■0.00	Zero quantity
4	-2	38	Error	Reject negative quantity
5	12.5	-38	Error	Reject negative rate
6	abc	38	Input error	Reject non-numeric input

For tests 1–3, the function returns the expected payment. Tests 4–6 should display an error instead of calculating a payment. For test 6, float('abc') raises ValueError and is handled by the except block.

TASK 10 — Compare Expectations with Execution

Input / condition	Expected behaviour	Actual program behaviour
12.5 litres at ■38	Show ■475.00	Calculates 12.5 × 38 and prints 475.00
10 litres at ■38	Show ■380.00	Calculates 10 × 38 and prints 380.00
Zero litres	Show ■0.00	Returns zero payment

Input / condition	Expected behaviour	Actual program behaviour
Negative quantity or ra t	Reject invalid value	Raises ValueError and prints an error
Non-numeric text	Show input error	float() raises ValueError; except block handle s it

The expected and actual behaviours agree for the listed cases. This comparison shows why testing normal, boundary, and invalid inputs is important. The program should be run in the target Python environment to confirm the results.

TASK 11 — Recommend and Defend the Final Design

The recommended design uses a separate calculate_payment() function and a main() function. This design is simple, readable, and suitable for a small co-operative. The calculation function can be tested independently and reused by other programs. Input validation prevents negative quantities and rates from producing misleading payments. Formatting the result to two decimal places makes the displayed amount easier to read.

Design choice	Reason
Separate calculation function	Makes logic reusable and easier to test.
Meaningful names	Improves readability and maintenance.
Input conversion with float()	Allows decimal quantities and rates.
Validation and exception handlin g	Provides clearer responses to invalid input.
Two-decimal output	Matches common currency display conventions.

TASK 12 — Re-evaluate and Adapt the Design

The design can be adapted if the co-operative needs additional features. For example, the program could record the farmer's name, store daily transactions, support different rates for different milk categories, calculate bonuses or deductions, and generate a daily payment report. Any new feature should be discussed with the co-operative and tested before use.

Possible future improvements

1. Add the farmer's identification number and name.
2. Read the approved rate from a configuration file instead of hard-coding it.
3. Store transactions in a CSV file or database.
4. Use Python's Decimal type for more precise currency calculations in a production version.
5. Add a receipt showing quantity, rate, date, and final amount.
6. Test the program with real operational requirements before deployment.

Conclusion

The payment calculator provides a clear method for calculating milk payments from quantity and rate. The implementation uses functions, input conversion, validation, and formatted output. Test cases cover ordinary, boundary, and invalid inputs, while comparison of expected and actual behaviour helps assess correctness. The modular design is easy to maintain and can be expanded to support records, receipts, and additional co-operative payment rules.